MANUAL CORPORATIVO DE GIT

Guíadearquitecturaderamas,nomenclaturaybuenasprácticas de versión

DIMATREK

Empresa: Dimatrek **Versión:**2.0 (Normativa Oficial) **Área:** Ingeniería & DevOps **Audiencia:**Onboarding Técnico

1. INTRODUCCIÓN Y PROPÓSITO

El presente manual establece el estándar oficial de control de versiones para el equipo de desarrollo de **Dimatrek**. El cumplimiento riguroso de estas directrices garantiza la trazabilidad del código, la integridad de los despliegues en entornos de producción y la eliminación proactiva de conflictos de integración (*merge conflicts*). Todo nuevo integrante del equipo debe adoptar estas prácticas desde su primer día de trabajo.

2. ARQUITECTURA DE RAMAS PRINCIPALES (RAMAS VIVAS)

Las ramas vivas representan la columna vertebral de nuestro repositorio. Son permanentes, protegidas y **NUNCA** deben eliminarse ni recibir commits directos.

main — PRODUCCIÓN

Contiene el código activo en el entorno de producción. Refleja exclusivamente versiones 100% estables, probadas y auditadas.

- **Protección estricta:** Direct Push bloqueado.
- **Origen de actualización:** Solo vía PR desde develop o hotfix/.
- **Despliegue:** Activa el pipeline de CI/CD automático.

develop — INTEGRACIÓN

Punto de encuentro diario para el equipo. Refleja las últimas características terminadas antes del próximo release.

- **Nexo central:** Todo desarrollo parte y regresa a esta rama.
- **Estabilidad:** Debe mantenerse compilable y funcional.
- **Merges:** Exclusivamente mediante Pull Requests aprobados.

3. RAMAS TEMPORALES Y NOMENCLATURA OBLIGATORIA

Las ramas de vida corta se crean para un propósito específico y deben eliminarse inmediatamente tras fusionarse.

PREFIJO	PROPÓSITO / USO	ORIGEN	DESTINO	EJEMPLO
🚀 **feature/**	Nuevas funcionalidades, componentes de UI o lógica de negocio.	develop	develop	**feature/login-validation**
📝 **docs/**	Documentación técnica, READMEs, especificaciones de API o diagramas. Prohibido incluir cambios de código.	develop	develop	**docs/actualizar-readme-api**
🚨 **hotfix/**	Corrección de errores críticos e imprevistos en el entorno de Producción.	main	main & develop	**hotfix/corregir-fuga-memoria**

4. FLUJO DE TRABAJO PASO A PASO (WORKFLOW OFICIAL)

Para mantener la consistencia del repositorio de Dimatrek, todo desarrollador debe ejecutar el ciclo de vida estandarizado:

1 Sincronización Local

Asegúrese de partir siempre de la versión más reciente de la rama de integración.

```
git checkout develop
git pull origin develop
```

2 Creación de la Rama Temporal

Utilice el prefijo adecuado según la tabla normativa.

```
git checkout -b feature/login-validation
```

3 Trabajo Local y Commits Atómicos

Realice los cambios pertinentes y registre progresivamente el avance.

```
git add src/auth/login.js
git commit -m "feat: implementar validacion en formulario de registro"
```

4 Publicación en Servidor Remoto

Suba la rama local hacia el repositorio centralizado de Dimatrek.

```
git push origin feature/login-validation
```

5 Apertura de Pull Request (PR) y Code Review

Abra un PR en la plataforma hacia `develop`. Requiere la revisión y **aprobación obligatoria de al menos 1 compañero** antes de consolidar la fusión.

6 Merge y Eliminación Automática

Tras la aprobación y el paso positivo de las pruebas automáticas, se efectúa el Merge con la opción *Delete branch* activada.

5. ESTANDARIZACIÓN DE COMMITS (CONVENTIONAL COMMITS)

Dimatrek exige mensajes de commit claros, semánticos y profesionales para la generación precisa de logs técnicos.

✅ Ejemplos Permitidos (Semánticos)

- feat: implementar validación en registro
- docs: actualizar guía de instalación de API
- fix: corregir error de renderizado en dashboard
- refactor: optimizar consulta de usuarios

❌ Ejemplos Prohibidos (Inaceptables)

- git commit -m "arreglos"
- git commit -m "ahora sí funciona"
- git commit -m "cambios varios"
- git commit -m "fix bug final final"

6. LAS 3 REGLAS DE ORO DE DEVOPS

🎯

Una Rama, Un Propósito

Prohibido mezclar refactorizaciones, cambios de lógica y documentación de áreas distintas en un mismo PR.

⚡

Ramas Cortas

Las ramas deben vivir pocas horas o días. Promueva la integración continua frecuente para evitar *merge conflicts*.

📜

Historial Claro

Cada commit debe ser atómico y descriptivo. Permite rastrear y revertir cambios en auditorías técnicas de forma aislada.

⛔ **ADVERTENCIA DE SEGURIDAD Y NORMATIVA DE DEPLOY**

Está estrictamente prohibido el uso de `git push --force` o `git push -f` sobre las ramas `main` y `develop`. Toda infracción deshabilitará automáticamente las credenciales de despliegue del desarrollador y requerirá una revisión del Lead DevOps.

7. TABLA DE REFERENCIA RÁPIDA (CHEAT SHEET DE GIT)

Comandos de uso frecuente en la operación diaria de ingeniería:

COMANDO	ACCIÓN / DESCRIPCIÓN
`git status`	Muestra el estado de los archivos en el directorio de trabajo y área de staging.
`git checkout -b <nombre-rama>`	Crea una nueva rama a partir de la actual y cambia a ella de inmediato.
`git fetch origin`	Descarga los últimos cambios del servidor remoto sin aplicarlos localmente.
`git pull origin develop`	Trae e integra los cambios remotos de `develop` a la rama actual.
`git log --oneline --graph`	Muestra el historial de commits simplificado en formato gráfico dentro de la terminal.
`git stash` / `git stash pop`	Guarda temporalmente los cambios no confirmados para limpiar el espacio de trabajo.
`git branch -d <nombre-rama>`	Elimina la rama local especificada (utilizar tras completar el Merge).

Aprobado por:
Comité de Arquitectura & DevOps Dimatrek

Código de Documento:
POL-DEV-GIT-2026-V2